

ECTE333

Lecture 2 - C Programming, Digital IO

Prof. Lam Phung

School of Electrical, Computer and Telecommunications Engineering
University of Wollongong
Australia

Introduction

- Students are assumed to have learnt C programming in the second year of their degree.
- Hence, we review only briefly major aspects of C programming here.
 - Structure of a C program for AVR microcontrollers
 - Data types and operators
 - Flow control in C
 - C functions
- In all lectures of ECTE333, C code examples will be used extensively.

ECTE333's schedule

Lecture (2h)	Tutorial (1h)	Lab (2h)
L1: Introduction to AVR Microcontrollers		
➔ L2: C Programming, Digital IO	Tutorial 1	Lab 1
L3: Serial Communication		
	Tutorial 2	Lab 2
L4: Interrupts, Timers		
	Tutorial 3	Lab 3
L5: Pulse Width Modulators		
	Tutorial 4	Lab 4
L6: Analogue-to-Digital Converters		
	Tutorial 5	Lab 5
L7: Microcontroller Applications		
		Lab 6

Lecture 2's sequence

2.1 Structure of a C program



2.2 Data types and operators



2.3 Flow control in C



2.4 C functions



2.5 Digital IO in ATmega16

2.1 Structure of a C program

- A C program typically has two main sections.
 - **Section #include:** to insert header files.
 - **Section main():** code that runs when the program starts.
- In this example, header file **<avr/io.h>** contains all register definitions for the selected AVR microcontroller.

```
#include <avr/io.h> // avr header file for all registers/pins
int main(void){
    unsigned char i; // temporary variable
    DDRA = 0x00;      // set PORTA for input
    DDRB = 0xFF;      // set PORTB for output
    PORTB = 0x00;     // turn ON all LEDs initially
    while(1){
        // Read input from PORTA, which is connected to the 8 switches
        i = PINA;
        // Send output to PORTB, which is connected to the 8 LEDs
        PORTB = i;
    }
    return 1;
}
```

C instructions and blocks

- C instructions
 - C instructions control the program flow.
 - They consist of keywords, expressions and other instructions.
 - An instruction ends with a semicolon.

DDRB = 0xFF; // set PORTB for output
- C blocks
 - A C block is a group of instructions enclosed by braces {}.
 - It is typically associated with **if**, **switch**, **for**, **do**, **while**, or functions.

```
while (1){
    // Read input from PORTA - connected to the 8 switches
    i = PINA;
    // Send output to PORTB - connected to the 8 LEDs
    PORTB = i;
}
```

C comments

- Comments are text that the compiler ignores.
- For a single-line comment, use double forward slashes:

DDRA = 0x00; // set PORTA for input
- For a multi-line comment, use the pair /* and */:

/* File: led.c
Description: Simple C program for the ATMEL AVR(ATmega16 chip)
It lets user turn on LEDs by pressing the switches on the STK500
board
*/
- Always use comments to make the program easy to understand.

Inserting assembly code into a C program

将汇编代码插入到C程序

- To insert assembly code into a C program, use the **asm** directive:

asm("assembly instructions");

时钟周期时延

- For example, the following code creates a delay of one clock cycle.

```
asm volatile("nop");
```

- Keyword volatile is used to prevent compiler from removing the C instruction.

Lecture 2's sequence

2.1 Structure of a C program

2.2 Data types and operators

2.3 Flow control in C

2.4 C functions

2.5 Digital IO in ATmega16

C operators

■ C has a rich set of operators:

- Arithmetic operators
- Relational operators
- Logical operators
- Bit-wise operators
- Data access operators
- Miscellaneous operators

2.2 Data types and operators

■ The main data types in C are

- **char**: 8-bit integer in AVR
- **int**: 16-bit integer in AVR
- **long int**: 32-bit integer in AVR

■ The above data types can be modified by keyword 'unsigned'.

```
char a; // range of values for a: -128, ..., 0, ..., 127
unsigned char b; // range of values for b: 0, 1, 2, ..., 255
unsigned long int c; // range of values for c: 0, 1, ..., 232 - 1
```

无符号 (非负整数)

■ Examples of variable assignment:

```
a = 0xA0; // prefix 0x means hexadecimal format
a = 0b11110000; // prefix 0b means binary format
b = '1'; // b stores ASCII code of character '1'
c = 2000ul; // c stores unsigned long integer 2000
```

0x 前缀 16 进制
0b 前缀 2 进制
0d 10 进制

■ In C, a variable or function must be declared **before** it is used.

Arithmetic operators

算术运算符

Operator	Name	Example	Description
*	Multiplication	x * y	Multiply x times y 乘
/	Division	x / y	Divide x by y 除
%	Modulo	x % y	Remainder of x divided by y x ÷ y 取余
+	Addition	x + y	Add x and y 加
-	Subtraction	x - y	Subtract y from x 减
++	Increment	x++ ++x	Increment x by 1 after using it x 加 前使用 Increment x by 1 before using it x 加 后使用
--	Decrement	x-- --x	Decrement x by 1 after using it Decrement x by 1 before using it
-	Negation	-x	Negate x 负

Relational operators 关系运算符

Operator	Name	Example	Description (Return a value of ...)
>	Greater than	$x > 5$	$x > 5$ 赋1 否则赋0 1 if x is greater than 5, 0 otherwise
>=	Greater than or equal to	$x >= 5$	1 if x is greater than or equal to 5, 0 otherwise
<	Less than	$x < y$	1 if x is smaller than y, 0 otherwise
<=	Less than or equal to	$x <= y$	1 if x is smaller than or equal to y, 0 otherwise
==	Equal to	$x == y$	1 if x is equal to y, 0 otherwise
!=	Not equal to	$x != 4$	1 if x is not equal to 4, 0 otherwise

Bit-wise operators 位运算符

- These operators are applied on individual bits of a variable or constant.

Operator	Name	Example	Description
~	Bit-wise complement	$\sim x$	将每一位从0切换到1, 或1切换到0 按位非
&	Bitwise AND	$x \& y$	Bitwise AND of x and y 按位与 用于清零
	Bitwise OR	$x y$	Bitwise OR of x and y 按位或 用于置1
^	Bitwise XOR	$x \wedge y$	Bitwise XOR of x and y 按位异或
<<	Shift left	$x << 3$	Shift x three bits to the left 左移3
>>	Shift right	$x >> 1$	Shift x one bit to the right 右移

Logical operators 逻辑运算符

- These operators are applied on logical variables or constants.

Operator	Name	Example	Description (Return a value of ...)
!	Logical NOT	$!x$	$x=0$ 赋1 否则赋0 1 if x is 0, otherwise 0
&&	Logical AND	$x \&\& y$	$x=y=1$ 赋1 否则赋0 1 if both x and y are 1, otherwise 0
	Logical OR	$x y$	$x=y=0$ 赋0 否则1 0 if both x and y are 0, otherwise 1

Data-access operators

- These operators are for arrays, structures, or pointers.
- We'll learn more about them when required.

Operator	Name	Example	Description
[]	Array element	$x[2]$	Third element of array x
&	Address of	$\&x$	Address of the memory location where variable x is stored
*	Indirection	$*p$	Content of memory location pointed by p
.	Member selection	$x.age$	Field 'age' of structure variable x
->	Member selection	$p->age$	Field 'age' of structure pointer p

Miscellaneous operators

Operator	Name	Example	Description
()	Function	<code>_delay_ms(250)</code>	Call a library function to create a delay of 250ms
(type)	Type cast	<code>char x = 3;</code> <code>(int) x</code>	x is 8-bit integer x is converted to 16-bit integer
?	Conditional evaluation	<code>char x;</code> <code>y = (x > 5) ? 10 : 20;</code>	This is equivalent to: <pre>if (x > 5) y = 10; else y = 20;</pre>



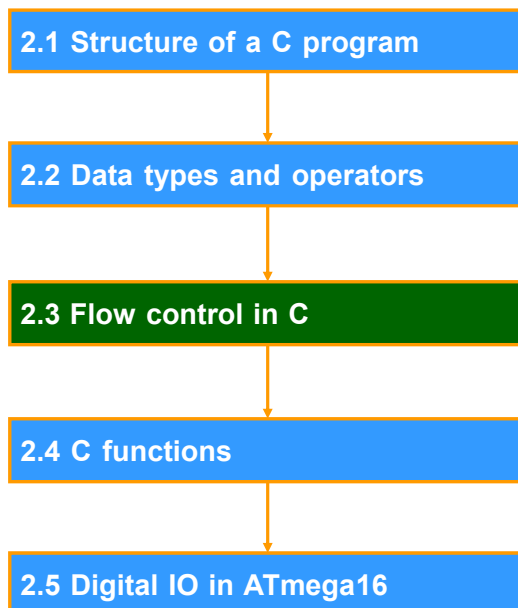
commonly used by C coders.

Atmel AVR demo: Tetris



A game console created by Uzebox from Atmel AVR

Lecture 2's sequence



2.3 Flow control in C

- By default, C instructions are executed sequentially.
- To change the program flow, there are 6 types of instructions.
 - if-else instruction
 - switch instruction

} **Conditional** 条件分支

 - while instruction
 - for instruction
 - do instruction

} **Iterative** 循环

 - goto instruction

} Should be avoided!

'if-else' instruction

General syntax

```
if (expression) 表达式(词组 例如 a > 0)
    instruction_1;
else
    instruction_2;
```

Example

1个字节8个bit 求和

```
char a, b, sum;
a = 4; b = -5;
sum = a + b;
if (sum < 0)
    printf("sum is negative");
else if (sum > 0)
    printf("sum is positive");
else
    printf("sum is zero");
```

*0x06 16进制 对应4个2进制
1100 1100*

'switch' instruction

General syntax

```
switch (expression)
    case constant_1:
        instruction_1;
        break;
    case constant_2:
        instruction_2;
        break;
    ...
    case constant_n:
        instruction_n;
        break;
    default:
        instruction_other;
}
```

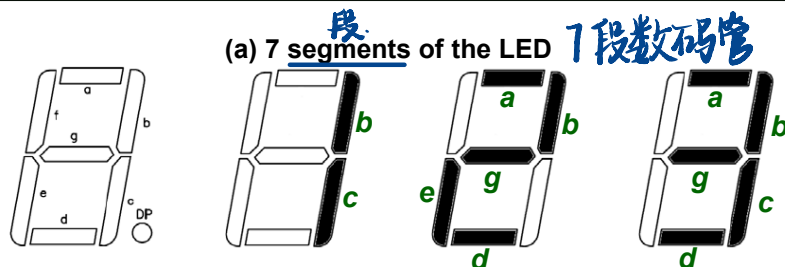
Use 'break' to separate different cases.

所有其它情况用默认

Use 'default' for all other cases.

'switch' instruction – Example

Lab 1: Find the bit pattern to display a digit on the 7-segment LED.



(b) Bit assignment on the 8-pin connector

LED segment	DP	g	f	e	d	c	b	a
Bit number	7	6	5	4	3	2	1	0
To display '1'	0	0	0	0	0	1	1	0
To display '2'	0	1	0	1	1	0	1	1

'switch' instruction – Example

```
unsigned char digit; // input: ASCII code of digit to display
unsigned char led_pattern; // output
switch (digit){
    case '0':
        led_pattern = 0b00111111;
        break;
    case '1':
        led_pattern = 0b00000110;
        break;
    case '2':
        led_pattern = 0b01011011;
        break;
    // You can complete more cases here...
    default:
}
```

2进制

LED segment	DP	g	f	e	d	c	b	a
Bit number	7	6	5	4	3	2	1	0
To display '1'	0	0	0	0	0	1	1	0
To display '2'	0	1	0	1	1	0	1	1

PORTB = led_pattern; // send to PORTB and 7-segment LED

'while' instruction

循环

■ General syntax

```
while (expression){  
    instructions;  
}
```

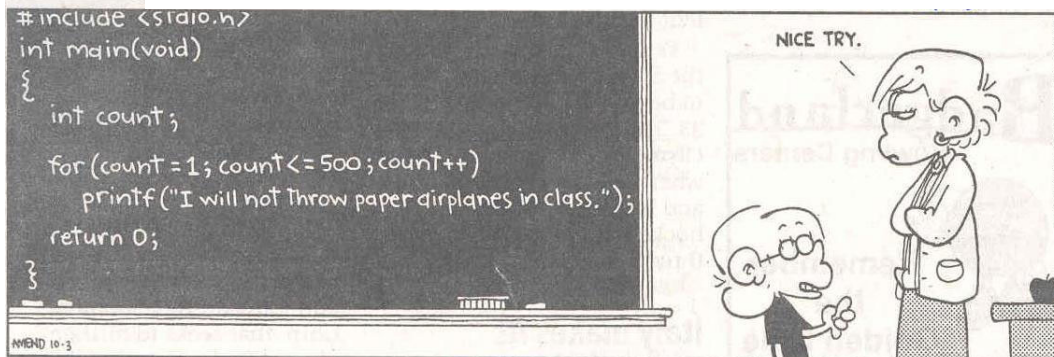
计算

■ Example: Compute the sum of $1 + 2 + \dots + 10$.

```
int sum, i;  
i = 1; sum = 0;  
while (i <= 10){  
    sum = sum + i; 满足条件才能进循环  
    i = i + 1;      "右边赋值给"左边的  
}                  /i++;
```

'for' instruction

FOXTROT



Write 500 times: "I will not ..."

'for' instruction

for 循环

■ General syntax

```
for (expression1; expression2; expression3){  
    instructions;  
}
```

- expression1 is run before the loop starts. 循环开始前运行
- expression2 is evaluated before each iteration. 每个循环开始前
- expression3 is run after each iteration. 循环后运行

■ Example: Compute the sum of $1 + 2 + \dots + 10$.

```
int sum = 0;  
  
for (int i = 1; i <= 10; i++){ 词组都放在for里  
    sum = sum + i;  
}
```

'do' instruction

■ General syntax

```
do {  
    instructions;  
} while (expression);
```

■ Example: Compute the sum of $1 + 2 + \dots + 10$.

```
int sum, i;  
i = 1; sum = 0; // Initialization  
do {  
    sum = sum + i;  
    i = i + 1;  
} while (i <= 10);
```

先执行一轮再操作

$sum = 1 + 2 + \dots + 10$

'break' instruction in loop

- The 'break' instruction inside a loop forces early termination of the loop.

提前结束

- What is the value of 'sum' after the following code is executed?

```
int sum, i;
i = 1; sum = 0; // Initialization
while (i <= 10){
    sum = sum + i;
    i = i + 1;
    if (i > 5)
        break;
}
```

sum = 1+2+...+5
当 i=6 时, 跳出循环

C arrays 数组

- An array is a list of values that have the same data type.
- In C, array index starts from 0.
- An array can be one-dimensional, two-dimensional, or more.

- This code example creates a 2-D array (multiplication table):

```
int a[8][10]; // 2维数组 (图像是二维数组)
for (int i = 0; i < 8; i++)
    for (int j = 0; j < 10; j++)
        a[i][j] = i * j;
```

- An array can be initialized when it is declared.

```
int b[3] = {4, 1, 10};
unsigned char keypad_key[3][4] = {{ '1', '4', '7', '*' },
                                   { '2', '5', '8', '0' },
                                   { '3', '6', '9', '#' }};
```

'continue' instruction in loop

跳过后续指令

- The 'continue' instruction skips the subsequent instructions in the code block, and forces the execution of the next iteration.

跳到执行下一次循环

- What is the value of 'sum' after the following code is executed?

```
int sum, i;
i = 1; sum = 0; // Initialization
while (i <= 10){
    i = i + 1;
    if (i < 5)
        continue;
    sum = sum + i;
}
```

和 sum = 5+6+...+10
break; 等价

Lecture 2's sequence

2.1 Structure of a C program

2.2 Data types and operators

2.3 Flow control in C

2.4 C functions

函数

2.5 Digital IO in ATmega16

2.4 C functions

- C functions are sub-routines that can be called from the main program or other functions.

子函数被其他函数调用
模块化设计

- Functions enable modular design, code reuse, and abstraction.

代码复用 抽象

- A C function can have a list of parameters and produce a return value.

参数列表 返回值

- Let's study C functions through three examples.

例子

Functions — Example 2

Write a function to compute the factorial n! for a given n:

$$n! = 1 \times 2 \times 3 \times \dots \times n$$

```
// factorial is the name of the custom function
// it accepts an input n of type int,
// it stores output at memory location by int pointer p
void factorial(int n, int* p) {
    int output = 1;
    for (int i = 1; i <= n; i++)
        output = output * i;
    *p = output; // store output at memory location pointed by p
}
```

返回值

```
int main(void) {
    int n = 5; // some example value of n
    int v; // variable to storage result
    factorial(n, &v); // call the function, store return value in v
}
```

传地址

Functions — Example 1

阶乘

Write a function to compute the factorial n! for a given n:

$$n! = 1 \times 2 \times 3 \times \dots \times n$$

```
// factorial is the name of the custom function
// it accepts an input n of type int, and return an output of type int
int factorial(int n) {
    int output = 1;
    for (int i = 1; i <= n; i++)
        output = output * i;
    return output; // return the result
}
```

这是一个for循环

```
int main(void) {
    int n = 5; // some example value of n
    int v; // variable to storage result
    v = factorial(n); // call the function, store return value in v
    return 1;
}
```

v=120

Functions — Example 3

Write a function to compute the factorial n! for a given n:

$$n! = 1 \times 2 \times 3 \times \dots \times n = (n-1)! \times n$$

```
// recursive function to compute the factorial of n
int factorial(int n) {
    if (n==0)
        return 1; // terminating condition
    else
        return n * factorial(n-1); // recursive call
}
```

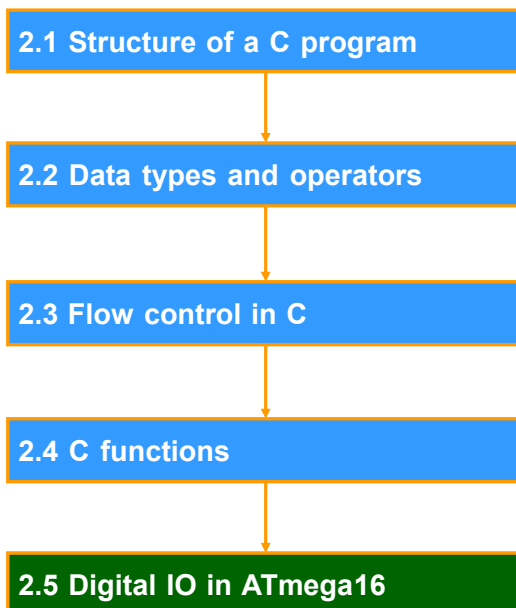
递归 ← 枚举

```
int main(void) {
    int n = 5; // some example value of n
    int v; // variable to storage result
    v = factorial(n); // call the function, store return value in v
    return 1;
}
```

Guidelines on C coding and documentation

- Write concise and efficient C code.
- Delete unnecessary lines of code.
- Format C code properly.
- Use correct indentation to show the logical structure of the program.
- Use meaningful names for variables and functions.
- Include concise C comments to explain code.
- Insert a blank line to separate code sections.
- For printing, use a fixed-width font such as `Courier New` for code.
- If a C instruction is too long, split it logically into multiple lines.
- Observe how C code is presented in the lecture/lab notes.

Lecture 2's sequence



Atmel AVR Demo: Graphics games



Atmel AVR Game Console by the Uzebox Project

2.5 Digital IO in ATmega16 数字IO口

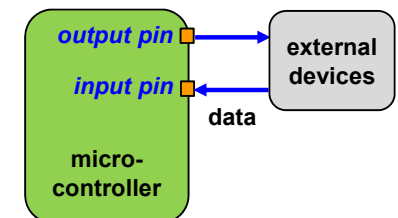
- ATmega16 has four 8-bit digital IO ports:

- PORT A,
- PORT B,
- PORT C, and
- PORT D.

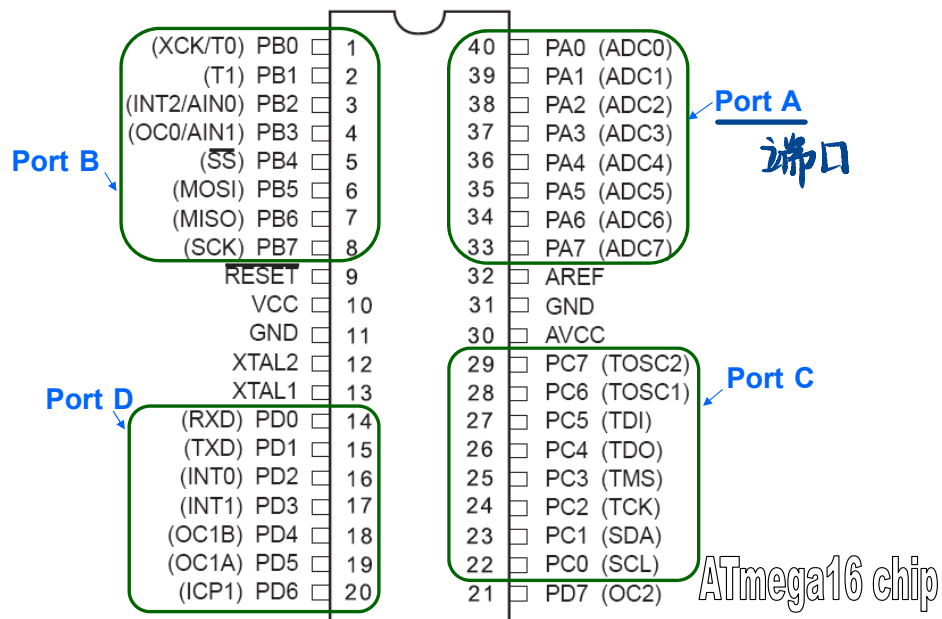
- Each port has 8 data pins.

- Every port is bi-directional.

- Each of the 8 pins can be individually configured for input or output.



Digital IO in ATmega16: Pins 引脚



Digital IO in ATmega16: Configuring for input/output

- For each port, there are three 8-bit registers.

- Data Direction Register (DDRx)
- Input Pins Address (PINx)
- Data Register (PORTx)

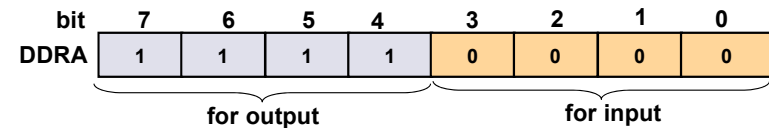
Here, x denotes A, B, C, or D.

数据流向寄存器

- Data Direction Register (DDRx) decides a pin is output (1) or input (0).

Example: To set Port A pins 0 to 3 for input, pins 4 to 7 for output,

```
DDRA = 0b11110000; // configure pins
```



Digital IO in ATmega16: Reading from/Writing to Port

- Data Register (PORTx) is used to write output data to port.

Example: To write to Port A 用于将输出数据写入端口

```
PORTA = 0b10111111; // write output
7 6 5 4 3 2 1 0
```

- Input Pins Address (PINx) is used to read input data from port. 读取输入数据

Example: To read the input pins of Port A

```
unsigned char temp; // temporary variable
temp = PINA; // read input
```

- Where do the C names PINA, PORTA, and DDRA come from?

```
// Header file <avr/iom16> ...
#define PINA _SFR_IO8(0x19)
#define DDRA _SFR_IO8(0x1A)
#define PORTA _SFR_IO8(0x1B)
```

AVR header file

- To access AVR microcontroller registers, your program must have:

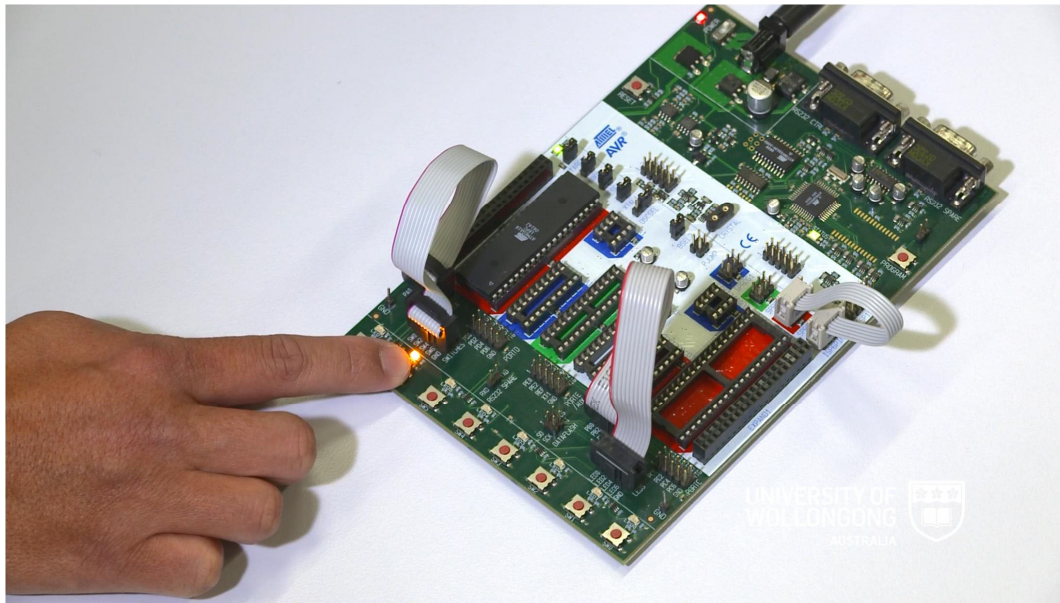
```
#include <avr/io.h>
```

- Depending on the Atmel AVR chip selected in your project, file 'io.h' redirects to a specific header file.

- Example**

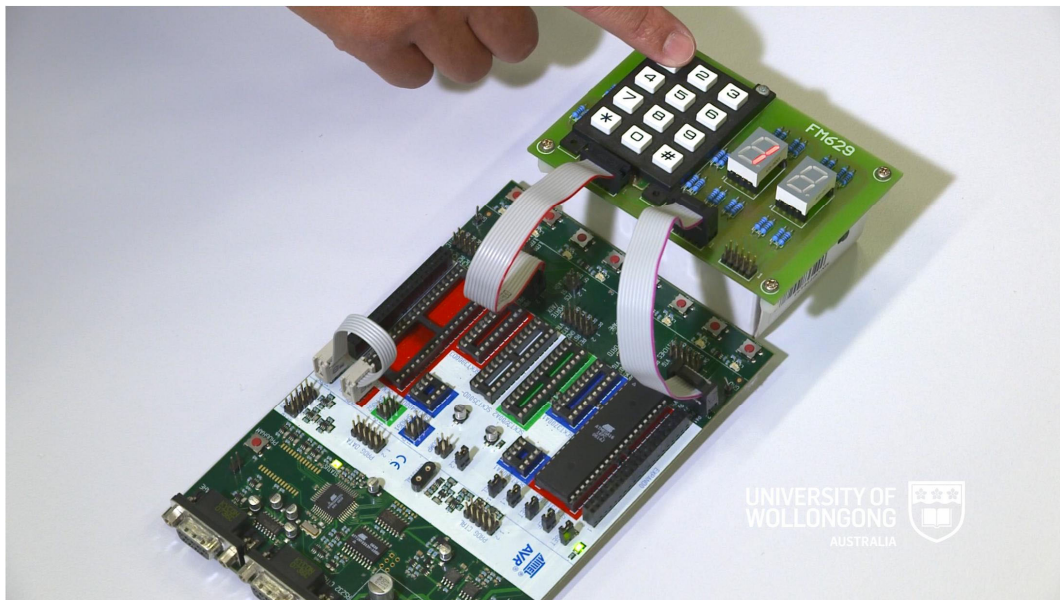
- For ATmega16 chip, the specific header file is 'avr/iom16.h'.
- This header file is in Appendix D of the lab notes.
- It lists the C names and addresses for all ATmega16 registers.
- We always use the C names in our code.

Digital IO in ATmega16: Example



Turning on LED by pressing a switch ([Lab 1, Tasks 1, 2, 3](#)).

Digital IO in ATmega16: Demonstration



Using Digital IO ports for 4x3 keypad and 7-segment display ([Lab 1, Tasks 4 and 5](#))

Digital IO in ATmega16: Example

```

/* File: led.c
Description: Simple C program for the ATMEL AVR uC (ATmega16 chip)
It lets user turn on LEDs by pressing the switches on STK500 board
*/
#include <avr/io.h> // AVR header file for all registers/pins
int main(void){
    unsigned char i; // temporary variable 临时变量

    DDRA = 0x00; // set PORTA for input
    DDRB = 0xFF; // set PORTB for output
    PORTB = 0x00; // turn ON all LEDs initially 最初打开所有LED

    while(1){ 无限循环
        // Read input from PORTA. 读入
        // This port will be connected to the 8 switches
        i = PINA; 在端口A读取数据并将其存储到i中 (开关状态)

        // Send output to PORTB. 输出
        // This port will be connected to the 8 LEDs 控制LED灯亮灭
        PORTB = i; 将i中的值输出到端口B

    }
    return 1;
}
    
```

Lecture 2's summary

■ Key points of this lecture:

- Basics about C programming.
- Programming the digital IO ports of ATmega16.

■ Next activities

- Tutorial 1: 'C programming for the AVR' .
- Lab 1: 'C programming for the AVR'
 - ❖ Study video demos of Lab 1.
 - ❖ Complete online Pre-lab Quiz for Lab 1.
 - ❖ Write programs for Tasks 4 and 5 of Lab 1.

Lecture 2's references

- M. A. Mazidi, S. Naimi, S. Naimi, AVR Microcontroller and Embedded Systems: Using Assembly and C, 2nd edition Pearson, 2015, [Chapter 7: AVR Programming in C, Chapter 4: AVR I/O Port Programming].
- J. Pardue, C Programming for Microcontrollers, 2005, SmileyMicros, [Chapters 1-6].
- S. F. Barrett and D. J. Pack, Atmel AVR Microcontroller Primer: Programming and Interfacing, 2008, Morgan & Claypool Publishers, [Chapter 1].
- D. V. Gadre, Programming and Customizing the AVR Microcontroller, 2001, McGraw-Hill, [Chapters 1-3].
- Atmel Corp., 8-bit AVR microcontroller with 16K Bytes In-System Programmable Flash ATmega16/ATmega16L, 2010. **Manual**